Measuring Security Hardening Adoption in Infrastructure as Code: An Empirical Study of Public Repositories

Ruining Yang
Department of Computer Science
Stony Brook University
Stony Brook, USA
ruiniyang@cs.stonybrook.edu

Nick Nikiforakis
Department of Computer Science
Stony Brook University
Stony Brook, USA
nick@cs.stonybrook.edu

***Abstract*—Security hardening is a critical practice for managing server infrastructure. Yet, despite the availability of many guides on how to harden servers, the actual adoption of these hardening steps is unknown. Infrastructure as Code (IaC) automates the setup, configuration, and management of servers in a reproducible and auditable way. In modern environments, IaC is the ideal medium through which to harden newly-provisioned servers since it would guarantee the consistent security hardening of all servers simultaneously.**

In this paper, we systematically collect and analyze the hardening guidance from 50 public security guides, translating it into IaC steps. Inspired by the term of "security smells" in software engineering, we refer to the absence of security-hardening steps as "ghost smells", i.e., steps that should be there yet are absent. Using Large Language Models, we analyze millions of public IaC playbooks across hundreds of thousands of GitHub repositories, to quantify the actual adoption of security hardening.

Overall we find minimal adoption of what are considered critical steps in hardening a new server. Approximately 10% of the repositories written in one of the most popular IaC languages take *any* hardening steps, with most IaC focusing primarily on securing Linux but largely ignoring the hardening of common server software. Our results indicate that ghost smells are a real issue in IaC and warrant inclusion in static analysis tools.

***Index Terms*—Infrastructure as Code, security hardening.**

I. Introduction

Cloud computing has fundamentally transformed how organizations manage infrastructure. Modern cloud environments routinely operate millions of servers distributed across data centers worldwide. This scale creates a massive attack surface where the services on each server may expose potential vulnerabilities through open network ports, poor authentication, broad file permissions, and poorly-chosen configuration parameters. The security community has established well-documented hardening practices to reduce that attack surface. Formal benchmarks such as the CIS Benchmarks [1] and NIST security checklists [2] provide detailed, parameter-level recommendations for securing operating systems, databases, and web servers. These guidelines specify concrete configuration changes such as how to disable root login via SSH. Downstream from these formal benchmarks, the security community maintains security checklists and "must-do" steps when installing and configuring a new server. Despite this extensive body of knowledge, the actual adoption of these measures in production systems remains largely unknown.

The need to maintain an ever-expanding cloud motivated the design of Infrastructure as Code (IaC). In IaC, organizations write machine-readable files to define and provision their infrastructure. This approach replaces manual system administration with code that specifies the desired state of servers, networks, and other components, making infrastructure changes reproducible and auditable. Tools such as Ansible and Puppet automate infrastructure management across thousands of machines simultaneously, eliminating inconsistencies and the human errors of manual configuration. As a result, a single IaC "playbook" can configure firewall rules on one thousand servers as reliably as on one server.

Although IaC offers significant advantages, it also concentrates risk. Infrastructure code exists in code repositories. A single misconfiguration, such as, an exposed network port or weak credentials, can now propagate instantly across all systems when the playbook executes. There is substantial prior work where authors identified insecure code patterns (commonly known as "security smells") in IaC and studied their presence across languages, codebases, and vulnerability domains [3]–[7]. What is common in prior work is that researchers first define what is an insecure code pattern (i.e. a security smell) and then use static analysis to search for that pattern automatically in large IaC datasets.

In this paper, we take a fundamentally different approach. Instead of searching for code patterns that should not be present, we instead search for the ones that should be present and yet are absent. Namely, we seek to understand whether the world of IaC is actually adopting the aforementioned battle-tested advice on how one should secure new servers and infrastructure. Inspired by the terminology of security smells, we define these patterns as *ghost smells*, i.e., secure patterns that are missing from an organization's IaC playbooks. These include the installation of a properly-configured firewall, the hardening of an SSH daemon, and the turning-on of known modules and options to secure web servers and databases.

These ghost smells do not violate syntax rules and produce no runtime errors. The infrastructure still deploys successfully but each missing hardening configuration leaves systems vulnerable to well-known attacks.

To this end, we start by turning server-hardening advice into a concrete rule catalog and then use that catalog to check real IaC code. We analyze 50 public server-hardening guides for some of the most popular server software, normalize different phrasings into the same advice, and keep only the advice that appears in multiple independent sources while dropping vague advice and guidance. Equipped with this catalog, we download and analyze millions of Ansible and Puppet playbooks from hundreds of thousands of public GitHub repositories in search for ghost smells (i.e. the lack of adoption of popular security configuration steps). We rely on Large Language Models (LLMs) to analyze IaC code written by a diverse set of developers and establish a set of tests to quantify which LLM performs the best for our study.

Among others, we discover that from the hundreds of thousands of repositories that include IaC code, only a small minority (10.81% for Ansible and 4.70% for Puppet) make use of at least one security-hardening rule or configuration. Within the secured repositories, Linux hardening dominates. In Ansible, regular system updates appear in 51.06% of secured repositories, while firewall configuration appears in 33.94%. Puppet shows a different emphasis, where firewall rules appear in 41.14% of secured repositories and regular updates appear in only 11.67%. Application hardening remains uncommon even among repositories that use these services. Steps to harden a MySQL server are observed the most, yet even then only 16.27% of Ansible and 7.09% of Puppet repositories that deploy MySQL make use of them. Contrastingly, web-server hardening is near absent from most IaC repositories. Lastly, we quantify the hardening of multi-service systems (e.g. MySQL *and* Linux). Among MySQL users, only 6.00% of Ansible repositories and 2.14% of Puppet repositories secure both Linux and MySQL, while the corresponding Linux plus web server combinations stay at or below 0.41%.

Overall in the domain where following security-hardening practices is the most important (i.e. in the playbooks that are used to create potentially thousands of identically configured servers), that is where those practices are sorely lacking. Therefore, expanding static analysis tools with ghost smell detection can substantially increase infrastructure security.

II. Background

A. Infrastructure as Code

Infrastructure as Code (IaC) has become a foundational practice in DevOps and cloud computing. Organizations write machine-readable files to define and provision their infrastructure. This approach replaces manual system administration with code that specifies the desired state of servers, networks, and other components. Ansible and Puppet are two widely used IaC tools [8], [9]. They automate infrastructure management but use different approaches. Ansible uses an agentless architecture with YAML playbooks where playbooks define

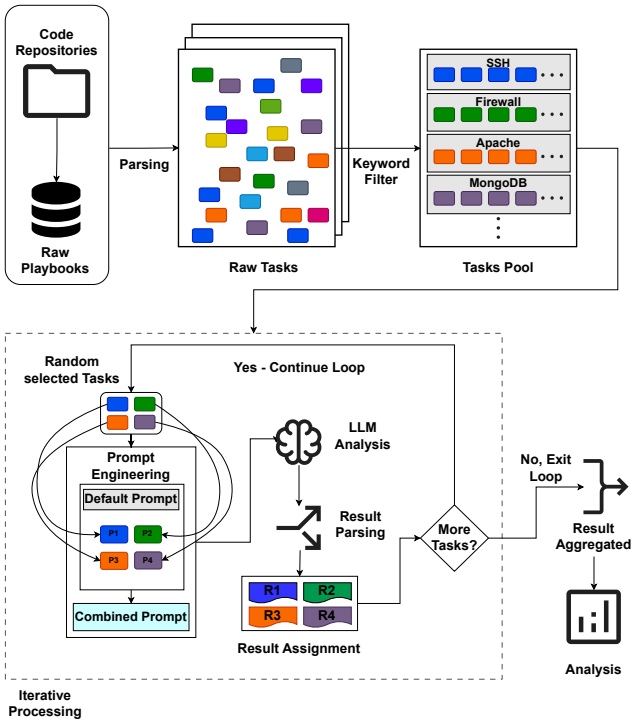

Fig. 1: *Security hardening rule detection pipeline*

tasks that execute on target machines. Contrastingly, Puppet uses an agent-based model with a Domain Specific Language (DSL), where master and agents maintain the desired system state. Both tools let teams automate deployments and maintain consistent configurations across many systems.

Although IaC offers significant advantages, it also introduces security risks. A core property of IaC is idempotency: scripts are designed to be run repeatedly against infrastructure to guarantee that a system always converges to the desired state. Therefore, even if a hardening step was applied manually or outside of IaC, it would still be prudent to include it in a playbook. A single misconfiguration in a script can propagate to all systems instantly [4]. Common errors include exposing network ports, using weak credentials, granting excessive permissions, and disabling security features. In large organizations, IaC repositories can contain thousands of files with dependencies across variables, roles, and templates. Manual security reviews cannot scale to this volume [10]. Therefore, automated analysis tools are necessary but face a challenge. They must understand not just syntax but the security intent of configurations. Traditional static analyzers rely on pattern matching and cannot recognize when different code structures achieve the same security goal.

B. Security Hardening Baselines

System hardening reduces a system's attack surface through removing unnecessary software and configuring system parameters. Organizations need to use security baselines to apply hardening consistently. A security baseline is a documented set of configuration requirements. It provides a standard for

measuring system security. Organizations deploy all systems with the same baseline to maintain consistent security.

Formal benchmarks provide detailed hardening guidance. The CIS Benchmarks [1] and OWASP guidelines [11] offer some parameter-level recommendations for securing operating systems, databases, web servers, etc. These benchmarks are widely adopted for compliance and auditing. The National Institute of Standards and Technology [2] maintains similar security configuration checklists for various technologies. Security practices also come from community knowledge. Practitioners share hardening recommendations through security guides and software documentation. This knowledge is less formal than benchmarks but reflects real world operational experience.

C. Large Language Models

Large Language Models (LLMs) are deep learning models trained on large datasets of text and source code. Current mainstream models include ChatGPT [12], Claude [13], Gemini [14], and DeepSeek [15]. Using the transformer architecture [16], these models learn to recognize and generate patterns in sequential data. Unlike traditional static analysis tools that use fixed grammatical rules, LLMs develop semantic understanding of their input [17]. They can infer the intent behind code rather than just verify syntax, recognize that different commands achieve the same functional outcome.

This capability is useful for analyzing IaC playbooks since a single security objective can be implemented in multiple ways, such as configuring firewalls using firewalld, ufw, or iptables modules, or installing packages via apt, yum, or dnf. A traditional static analyzer needs a separate rule for each variant, making its rule set incomplete. LLM can generalize across these syntactic differences to identify the underlying intent, as research shows LLMs can understand semantically equivalent code implementations even when syntax differs [18].

Users interact with LLMs through prompts, which are text queries that guide the model to generate the desired output, and excellent prompt design significantly impacts model performance [19]. Currently, LLMs have two practical constraints for analyzing code at scale. First, they have a finite context window measured in tokens. For example, GPT-4 Turbo has a token limit of 128k, which is approximately 96,000 words [20], [21]. A typical IaC file could contain hundreds to thousands of lines and a large repositories contain many files. Processing all code in one request is not feasible. Orthogonal to the context-window limitations, commercial LLM APIs charge per token, which could make the file-by-file analysis of large codebases prohibitively expensive. Our analysis pipeline addresses both limitations through task level batching, as described in Section III.

III. System Design

Figure 1 shows our pipeline to measure adoption of known hardening practices in real-world IaC playbooks using LLMs. Our methodology has three main stages. First, we mine public security guides to build a catalog of security hardening rules. Second, we collect and preprocess IaC repositories from GitHub. Third, we batch the resulting tasks for analysis.

A. Identification of Common Protection Mechanisms

In this section, our first design step is to build a catalog of security hardening rules for five common software components: Linux, Apache, Nginx, MySQL, and MongoDB. Our goal is to identify server-hardening best practices that can be expressed as configuration parameters in IaC playbooks. Using the Google search engine, we collected the first ten results for the query "how to secure X" where X is each of our aforementioned common server components. From each webpage, we extracted all security recommendations that involve configuration settings. We did not compile this list based solely on CIS, NIST, or OWASP, as our goal was to track the recommended content that users most frequently come across online, rather than to study formal benchmark specifications.

Different websites describe the same rule in different ways. Some provide command-line examples with explanations. Others describe the rule in prose, without providing code samples. We normalized these variants into a single rule entry. For example, recommendations about disabling root login via SSH all map to the `PermitRootLogin no` setting, regardless of how the original source phrased it.

After normalization, we applied two filters. First, we kept only recommendations that appeared on multiple independent websites. Second, we excluded vague guidance like "minimize installed packages" because it depends on the deployment context and cannot be expressed as a single parameter setting. Examples of rules we kept include removing MySQL default accounts, disabling Nginx ServerTokens, and changing the SSH's daemon default port (TCP/22) to reduce credential brute-forcing attacks. The final catalog contains rules that can be written directly as Ansible tasks or Puppet resources. Table I lists our resulting configuration recommendations along with the number of independent sources.

B. Evaluating LLMs on Security-Related Tasks

We test whether LLMs can understand the semantics of IaC tasks in the context of our security hardening rules. We consider both proprietary models (e.g., ChatGPT and Claude) and open source models (e.g., Llama and Gemma) to compare their capability in security hardening rule recognition. Our goal is to test whether different models can correctly understand the meaning of each task.

To avoid false positives from keyword matching, we created three types of test cases for each security hardening rule. First, positive cases where the task correctly implements the rule. For example, setting the SSH port to a non-default value. Second, almost correct cases that look related but do not complete the hardening. For example, setting the SSH port to 22 or only modifying comments. Third, negative cases that are in the same domain but unrelated to the rule. For example, restarting the sshd without changing any security parameters.

TABLE I: *Security hardening rule catalog with source counts*

Rule	Count
Linux	
Set up firewall	10
Ensure the system regular updates	9
Use 2FA	7
Install Fail2ban	7
Change SSH port	6
PermitRootLogin NO	6
PasswordAuthentication NO	5
Configuring automatic updates	4
Use logwatch to monitor logs	2
MySQL	
Change default port mappings	7
Remove default accounts	5
Remove test databases	5
Disable history file	3
Disable local INFILE	2
MongoDB	
Enable authorization	7
Connect over SSL	5
Change default port	4
Bind IP only to localhost/private IP	3
Nginx	
Disable ServerTokens	8
Implement strong TLS	6
Enable HTTP Strict Transport Security (HSTS)	5
Enable X-XSS Protection header	5
Use ModSecurity	5
Limit concurrent connections by buffer size	5
Enable X-Frame-Options header	4
Enable Rate Limiting	4
Apache	
Disable ServerTokens	5
Disable directory listing	5
Limit CGI	4
Use ModSecurity	4
Disable .htaccess	3
Disable FollowSymLinks	3

For Ansible, we included multiple ways to implement the same rule. For example, changing the SSH port can use lineinfile, replace, blockinfile, or shell commands. For Puppet, we covered common patterns using resource attributes and template paths. We randomized variable names and removed comments from all examples to prevent the model relying on contextual clues instead of understanding the task itself. We use LLMs rather than static analysis because in IaC scripts, the same hardening operation can be written in multiple ways, and static analysis is more likely to either trigger false positives or fail to accurately identify the intent of the task.

We then used one prompt template for all tests. The prompt lists all security hardening rules we gathered from Section III-A and asks the LLM to only answer Yes or No for each rule. Example formats are `Linux - Set up firewall: Yes/No` or `Linux - Ensure regular system updates: Yes/No`. The prompt states that the model should answer Yes only if the task makes a parameter-level change consistent with the rule. Tasks that set parameters to default values must be labeled No. We included multiple examples in the prompt for each security hardening rule and set LLM "temperature" to zero for deterministic outputs. We also manually reviewed the LLM's results for 200 sample tasks before full deployment and found no misclassifications.

C. Collecting IaC Repositories

In order to analyze as many real world scripts as possible, we collected IaC code repositories by searching GitHub using the keywords "ansible" and "puppet." For Puppet, we made use of GitHub's `language:puppet` filter to include repositories that use Puppet but do not mention the term in their metadata. We collected Ansible repositories from March 2012 to May 2025 and Puppet repositories from January 2010 to July 2025. This timeframe was chosen to include the entire lifecycle of both tools. Puppet repositories appear on GitHub from early 2010, whereas Ansible emerges in 2012. Our goal was to cover as many Ansible and Puppet scripts as possible, so we did not exclude older repositories as scripts from several years ago remain downloadable and executable. We excluded forked repositories but did not distinguish between repositories intended for production, tutorials, or examples. Given the nature of public GitHub content, such a mix is inevitable, so we limit our scope to publicly available IaC material. Our analysis is performed on the latest commit of every repository, so we do not investigate longitudinal aspects of IaC hardening.

After downloading the repositories, we focused on Ansible .yml or .yaml files and Puppet .pp files that contain executable tasks. Ansible YAML files can be playbooks or variable definition files. We applied two filters to keep only playbooks. First, we excluded paths used for variables, such as `group_vars/` and `host_vars/`. Second, we ran Ansible's syntax checker (`ansible-playbook -syntax-check`) on the remaining files. We discarded files that were not valid playbooks or contained parsing errors. For Puppet, we kept all .pp files because we found no reliable, non-heuristic way to separate variable files from executable manifests.

Given the prevalence of variable references in configuration tasks, we normalized variables before analysis. For Ansible, we used GASEL [22] to replace variables with the actual values. This removes differences caused by variable names while keeping the code valid. For Puppet, we built our own substitution tool, replacing variables with their assigned values if both appear in the same file. For example, if `$port = 8080` has already been declared, then every subsequent use of `$port` will be replaced with `8080`. We did not resolve variables that depend on runtime facts or external data, such as `$facts['os']['family']`. Our normalization runs statically without executing any code. We likewise did not resolve hardening measures implemented through templates or external configuration files.

D. Analysis

After normalizing the downloaded Ansible and Puppet files as described in Section III-C, we prepared them for LLM evaluation. Since each IaC script can contain thousands of lines, sending entire files to LLM APIs may exceed token limits and incur a prohibitively high cost, so we opted to analyze tasks instead of files. Each task is one unit of work, such as installing a package, or changing a configuration parameter. Figure 1 shows our complete analysis pipeline.

As each Ansible or Puppet file can contain multiple tasks, we built parsers to extract tasks from the normalized files (top left in Figure 1). For Ansible, we filtered out tasks that do not perform actual operations, such as `include_tasks` and `include_role`. These tasks only load other files and do not change system configuration. We also removed comments and tags from each task since neither affects what a task does, but they increase token usage when sent to the LLM. For Puppet, we applied the same process. We then deduplicated the extracted tasks to create two task libraries, one for Ansible and one for Puppet. We categorized each task by its function using keywords ("Keyword Filter" in Figure 1). For example, tasks related to Apache usually contain keywords such as "apache2" and "httpd." Tasks related to firewalls usually contain "firewall," "firewalld," "ufw," and "iptables." We created categories for MySQL, MongoDB, Apache, and Nginx. For Linux tasks, we created finer grained categories such as SSH, firewall, and two-factor authentication. In total, we defined 12 categories.

The key innovation of our pipeline is how we reduce API costs while maintaining precise result attribution ("Iterative Processing" loop in Figure 1). A naive approach would send each task individually to the LLM, requiring one API call per task. However, it would result in very high costs if there are thousands or even millions of tasks. Instead, we grouped tasks into batches of four for LLM evaluation. Each batch contains tasks from different categories. For example, one batch might contain one SSH task, one Apache task, one MySQL task, and one Nginx task. When the LLM analyzes a batch, it answers "Yes" or "No" for each security hardening rule based on our prompt. For example, the LLM might answer "`Linux - Set up firewall: Yes`." Because each batch contains only one task per category, we know exactly which task the answer refers to ("Result Assignment" in Figure 1). If the LLM answers Yes to two MySQL security hardening rules, we know that the single MySQL task satisfies both rules. We can trace every positive answer back to exactly one task without ambiguity. This batching approach reduces API calls by over 70% compared to individual task analysis while preserving complete traceability. To reduce costs further, the prompt only asks about categories present in the batch. For example, if a batch does not contain any Apache tasks, the prompt does not include any Apache-related security hardening rules.

Some tasks belong to multiple categories. For example, a task that configures MySQL firewall rules belongs to both firewall and mysql. We chose to handle these tasks separately. To avoid ambiguity, we sent them individually to the LLM along with their corresponding prompts. This strategy reduces costs significantly compared to sending entire files with the full prompt to the LLM. After the LLM processes all batches ("Result Aggregated" in Figure 1), we aggregate results at the repository level for analysis. When analyzing our results, we consider a rule to be present if it appears at least once in the repository.

IV. Results

The following research questions drive the results presented in this section: **RQ1**: Hardening guidance is across many documents, and the same recommendation is often phrased in different ways. Can this advice be systematically collected and distilled it into a set of common security hardening rules? **RQ2:** Can we minimize LLM costs without sacrificing analysis pipeline fidelity? **RQ3**: What is the actual adoption rate of security hardening rules in public IaC repositories?

A. Common Security Hardening Rule Catalog

Table I shows the security hardening rules we extracted. The Count column indicates how many independent sources recommended each rule. We organize the catalog by component: Linux, MySQL, MongoDB, Nginx, and Apache.

Linux rules. Firewall configuration is the most recommended rule, appearing in all ten sources. Sources recommend installing a firewall with a default deny policy and allowlisting only required ports. Regular system updates appear in nine sources. Two factor authentication and Fail2ban each appear in seven sources, with Fail2ban blocking IP addresses after a threshold of failed authentication attempts.

Three SSH hardening rules also appear frequently. Six sources recommend changing the default SSH port and disabling root logins. Five sources recommend disabling password-based authentication in favor of key-based authentication. All three rules are expressed via single parameter settings in the SSH daemon's default configuration file. Automatic security updates appear in four sources and use the unattended-upgrades package with timer configuration. Logwatch appears in two sources for monitoring system logs and sending daily summaries.

Web server rules. For Nginx, the advice that we collected across guides and articles consists of turning off modules that unnecessarily leak server version information, ensuring strong TLS ciphersuites, adopting standard web-security mechanisms, such as X-XSS-Protection and X-Frame-Options headers, and using ModSecurity for rate limiting. While there was some overlap between Nginx and Apache, we found that Apache advice is focused more on the server-side configuration, as opposed to turning-on client-side security mechanisms.

Database rules. For MySQL, the high frequency items are changing the default port, which appears in seven sources. Removing default accounts and Removing the test database appears in five sources. The remaining advice is focused on disabling the client history file and the local INFILE feature which attackers could abuse to read files from the filesystem.

For MongoDB, enabling authorization appears in seven sources and this could be done by enable the authorization parameter in `mongod.conf`. Requiring TLS or SSL appears in five sources through the `requireTLS` parameter. Changing the default port appears in four sources. Binding only to localhost or private addresses appears in three sources. All MongoDB rules can be applied by changing parameters and can therefore be easily expressed in IaC tasks.

TABLE II: Comparison of GPT, Claude, Llama and Gemma test results

Test	Length	GPT	Claude	Llama	Gemma
1	Short	Y	Y	Y	Y
2	Short	Y	Y	N-FP	N-FP
3	Short	Y	Y	Y	Y
4	Short	Y	Y	Y	Y
5	Short	Y	Y	Y	Y
6	Medium	Y	P-MR	P-MFP	P-MR
7	Medium	Y	Y	P-FP	Y
8	Medium	Y	Y	N-FP	Y
9	Medium	Y	P-FP	P-FP	P-FP
10	Medium	Y	Y	N-FP	N-FP
11	Long	Y	Y	P-MFP	P-FP
12	Long	Y	Y	N-FP	N-FP
13	Long	Y	Y	P-FP	P-FP
14	Long	Y	Y	P-FP	P-FP
15	Long	Y	Y	P-FP	P-FP

Notes: Y = Pass; N = Fail; P = Partial pass; MR = Miss at least one rule; FP = False positive at least one rule; MFP = Miss and false positive at least one rule

B. Evaluating LLM in Security Hardening Rule Detection

To understand which LLM would be the best candidate for analyzing the adoption of security practices in real-world IaC code, we evaluate four popular LLMs using 15 Ansible test cases. Table II shows the results. We group the cases by task length: short (cases 1-5), medium (cases 6-10), and long (cases 11-15). We also classify them by expected outcome: positive cases correctly implement a security hardening rule (e.g., changing SSH port to a non-default value), while negative cases are in the same domain but do not implement any security hardening rules (e.g., restarting sshd service without configuration changes or setting parameters to their default value). By correctness, nine cases are positive (1, 3, 6, 7, 9, 11, 13, 14, 15) and six are negative (2, 4, 5, 8, 10, 12). Case 10 is an "Almost Correct" decoy to stress-test the semantic understanding of each LLM. The test set covers Linux (7 cases), MySQL (3 cases), MongoDB (1 cases), Nginx (2 cases), Apache (1 cases), and one mixed deployment case (12). We use an exact match criterion. To pass, a model must output the correct Yes/No for every rule in the list.

GPT passes all 15 cases. Claude passes 13/15 with two partial failures. Unfortunately, open-source models performed poorly. Gemma passes 6/15 and Llama passes 4/15 with both models exhibiting partial passes and false positives. By length, GPT passes 5/5 tests for all three groups. Claude also passes 5/5 for short and long cases but only 3/5 for medium cases. Llama passes 4/5 short cases and 0/5 medium cases, while Gemma passes 4/5 short cases and 2/5 medium cases

Claude has two partial failures, both on Linux cases. In case 6, a positive example, Claude misses a required rule (P-MR), while in case 9, Claude identifies the correct rule but adds a false positive (P-FP). Case 9 contains only automatic updates and does not include regular updates. The prompt examples distinguish these two concepts, but Claude confuses them.

Beyond the positive and negative cases, we designed an almost correct decoy containing similar parameter names, incorrect parameter values, irrelevant tasks and misleading commented. GPT and Claude correctly identify this as ineffective. Llama and Gemma make false positives on this trap.

Six positive cases (7, 9, 11, 13, 14, 15) implement multiple security hardening rules through different modules. GPT passes all 6. Claude shows one false positive in case 9, while Llama and Gemma fail on nearly all, showing that small models cannot unify semantically equivalent implementations.

In summary, when a rule is expressed as a single parameter pair, GPT and Claude achieve exact matches reliably. When a rule requires semantic consolidation or distinguishing parameters content, Claude occasionally makes errors. Llama and Gemma fail on these cases. GPT passes all 15 test cases which is why we choose it for the analysis of real-world IaC code.

TABLE III: Dataset summary for Ansible and Puppet repositories

Details	Ansible	Puppet
Number of Github repos	256,041	34,473
Number of GitHub files	4,973,095	526,528
Number of files tasks	15,426,245	1,665,557
Number of processed tasks	704,714	58,083
Number of unique repos with rules	27,681	1,619
Number of unique files with rules	52,976	2,929

C. Security Hardening Rules in Public Repositories

1) Dataset and Overall Coverage: We quantify real world security hardening rules adoption by analyzing GitHub repositories. Table III shows our dataset. We collected 256,041 Ansible repositories containing 4,973,095 script files and 34,473 Puppet repositories containing 526,528 script files.

After filtering as described in Section III-C, we extracted 704,714 tasks from Ansible playbooks and 58,083 tasks from Puppet that matched our security-related keywords. After deduplication, we obtained 199,881 unique Ansible tasks (28.36% of total) and 16,557 unique Puppet tasks (28.51% of total). This deduplication reduces LLM evaluation costs substantially, as we only need to analyze approximately one third of the extracted tasks.

The distribution of unique tasks across categories is uneven. For Ansible, system-update tasks dominate with 75,857 tasks, followed by firewall tasks with 63,841 tasks and MySQL-related tasks with 37,785 instances. The remaining 9 categories combined account for 22,398 instances. This distribution determined our batching strategy. We grouped tasks into four categories per batch: system update, firewall, mysql, and a combined category containing all other services. For Puppet, firewall tasks are even more dominant with 12,768 tasks, while the second largest category is mysql and its tasks appear only 990 times. It means that although we use the same method to query the LLM, in most cases only one potentially security task related to the firewall will be sent to the LLM. Under the prompt format in our study, we only spent about $300 on API token fees for Ansible playbooks, whereas the naive approach would have cost us thousands of dollars.

We validated our keyword based filtering by examining the most frequent tasks. The top five

TABLE IV: Repositories by service in Ansible and Puppet

Service	Ansible	Puppet
Linux	256,041	34,473
MySQL	24,901	4,723
MongoDB	5,090	764
Apache	31,726	4,889
Nginx	30,577	3,105

Ansible tasks are `apt: update_cache: yes`, `apt: update_cache=yes`, `service: name=iptables state=restarted`, `ufw: state: enabled`, and `ansible.builtin.apt: update_cache: yes`. All implement security-related operations. Similarly, the top five Puppet tasks all relate to the firewall rules confirming that our keyword filtering effectively isolates security-related tasks.

To accurately assess security hardening rule adoption rates, we first determined the baseline prevalence of each service in the dataset. Table IV shows the number of repositories using each service, regardless of whether they implemented security hardening rules. For both Ansible and Puppet, the number of repositories including MongoDB-related directives, is an order of magnitude lower than the repos for the rest of the services.

After LLM evaluation, we identified 27,681 Ansible repositories containing at least one security hardening rule across 52,976 files. For Puppet, 1,619 repositories contain security hardening rules across 2,929 files. This represents 10.81% of Ansible repositories and 4.70% of Puppet repositories in our dataset implementing at least one security hardening rule. The following sections analyze which specific rules appear and how often multiple services are secured together.

Table V shows security hardening rule adoption rates across service types. The coverage column indicates the percentage of repositories using each service that implement at least one security hardening rule for that service. MySQL shows the highest security hardening rule adoption rate in both platforms. Among Ansible repositories that use MySQL, 16.27% implement security hardening rules. For Puppet repositories that use MySQL, 7.09% implement these rules. Linux security shows moderate adoption with 9.67% of Ansible repositories and 3.82% of Puppet repositories implementing *any* of our identified hardening rules.

In contrast, web server security shows minimal adoption. For Apache, only 0.89% of Ansible repositories and 1.15% of Puppet repositories implement security hardening rules. Nginx adoption is even lower at 0.73% for Ansible and 0.93% for Puppet. MongoDB shows similarly low rates with 5.03% adoption in Ansible and 1.57% in Puppet.

2) Single Service Repositories: In Figure 2, we analyze how often each security hardening rule appears in repositories. We examine 27,681 Ansible repositories and 1,619 Puppet repositories that contain at least one security hardening rule.

Linux rules. Regular system updates and firewall configuration are the most widely adopted rules. In Ansible, system updates appear in 51.06% of repositories with security hardening rules, while firewall rules appear in 33.94%. Puppet shows different priorities: firewall rules appear in 41.14% of repositories while system updates appear in only 11.67%. Update automation also diverges across platforms. Automatic security updates appear in 24.27% of Puppet repositories compared to only 6.86% in Ansible, suggesting Ansible users prefer manual update control while Puppet users adopt automated mechanisms more readily.

TABLE V: Service type coverage comparison for Ansible and Puppet

Details	Service	Repos	Files	Coverage(%)
Ansible (27,681)	Linux	24,748	46,088	9.67
	MySQL	4,051	6,086	16.27
	MongoDB	256	456	5.03
	Apache	282	344	0.89
	Nginx	223	295	0.73
Puppet (1,619)	Linux	1,318	2,301	3.82
	MySQL	335	360	7.09
	MongoDB	12	60	1.57
	Apache	56	162	1.15
	Nginx	29	48	0.93

SSH rules show some adoption in Ansible: `PermitRootLogin` appears in 9.74% of repositories and `PasswordAuthentication` in 9.51%. Fail2ban, as a service typically adopted in the context of protecting SSH authentication, has a usage rate of 9.55%, which is similar to that of the other two SSH settings. Puppet shows lower rates at 6.24%, 3.09% and 7.23% respectively. However, other security hardening rules show drop-offs on both platforms. Changing the default SSH port appears in only 2.91% of Ansible repositories and 0.56% of Puppet repositories. Two-factor authentication, one of the strongest SSH controls in our catalog, appears in only 0.47% of Ansible repositories and 1.30% of Puppet repositories. Overall, we observe that IaC developers do implement basic SSH hardening measures but do not adopt more in-depth defensive measures.

Database rules. For MySQL, removing default accounts (13.37% in Ansible, 19.33% in Puppet) and test databases (12.30% in Ansible, 2.53% in Puppet) shows the strongest adoption. However, other MySQL rules show almost zero adoption. Only 0.03% of Ansible repositories change the default MySQL port, and only 0.01% disable the history file. MongoDB shows similarly low adoption across all rules, with the highest rate reaching only 0.43% in Ansible. This pattern suggests that developers implement some well-known MySQL hardening steps but overlook less common controls. MongoDB's low adoption may reflect its smaller user base compared to MySQL in the wild.

Web server rules. All Apache and Nginx rules appear in fewer than 2% of repositories on both platforms. In Ansible, the most common rules are disabling ServerTokens (0.44% for Apache, 0.35% for Nginx) and ModSecurity (0.39% for Apache, 0.22% for Nginx). In Puppet, adoption rates reach 1.67% for Apache ModSecurity and 1.05% for Nginx HSTS. Even widely recommended controls like strong TLS configura-

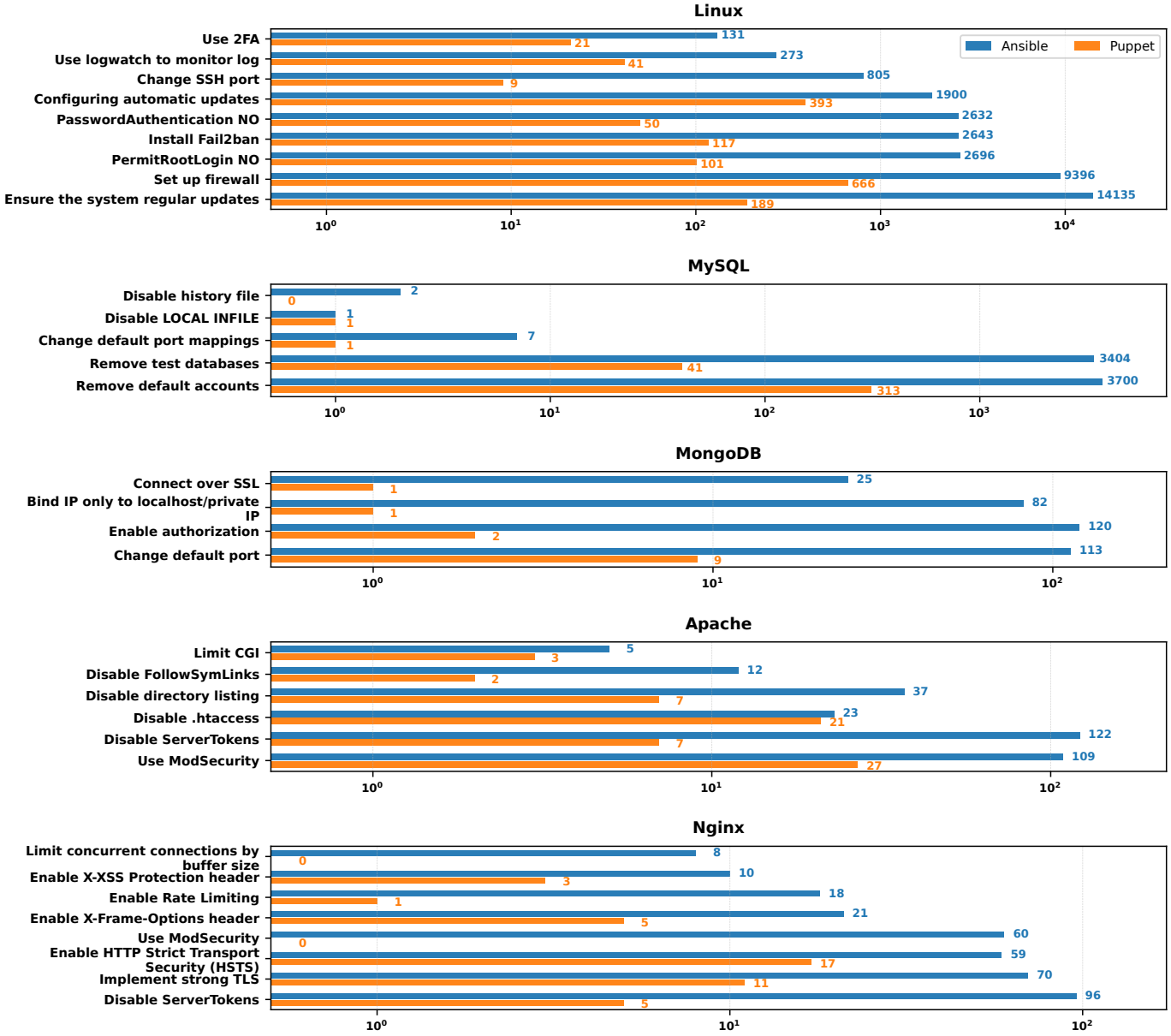

Fig. 2: *Security hardening rule adoption rates in Ansible and Puppet repositories*

tion appear in fewer than 0.68% of repositories, demonstrating a significant gap between security best practices and actual implementation despite these services being public facing.

3) Multi-services Repositories: We define multi-service coverage as a repository containing security hardening rules for at least two different service categories (See Table VI). Most repositories only secure one service: 25,934 of 256,041 Ansible repositories (10.13%) and 1,490 of 34,473 Puppet repositories (4.32%) implement one security hardening rule.

Linux and MySQL is the most popular multi-service combination. Among the 24,901 Ansible repositories that use MySQL, 1,494 (6.00%) implement both Linux and MySQL security hardening rules. Among the 4,723 Puppet repositories that use MySQL, 101 (2.14%) implement this pattern. MySQL shows relatively higher multi-service adoption than other services, likely because repositories that implement well-known MySQL hardening steps are doing so in the context of setting up a LAMP stack. Even then, the vast majority of MySQL users *do not* secure both layers together. MongoDB shows similar but lower rates. Among the 5,090 Ansible repositories that use MongoDB, 105 (2.06%) implement both Linux and MongoDB security hardening rules. Among the 764 Puppet repositories, only 2 (0.26%) implement this pattern.

Web server multi-service security remains extremely rare. Among Apache users, only 0.41% of Ansible repositories (130 of 31,726) and 0.41% of Puppet repositories (20 of 4,889) implement both Linux and Apache security hardening rules. Among Nginx users, only 0.33% of Ansible repositories (100 of 30,577) and 0.23% of Puppet repositories (7 of 3,105) secure both Linux and Nginx. These rates are notably low

TABLE VI: Service combinations in Ansible and Puppet repositories

Combination	Ansible Repos	Puppet Repos
Linux + MySQL	1,369	99
Apache + Linux	78	18
Linux + Nginx	69	7
Linux + MongoDB	53	2
Linux + MongoDB + MySQL	48	0
Apache + Linux + MySQL	45	2
Linux + MySQL + Nginx	28	0
Apache + MySQL	26	0
MongoDB + MySQL	12	0
MySQL + Nginx	8	0
Apache + Linux + MongoDB + MySQL	3	0
Apache + MongoDB	3	0
Apache + Linux + Nginx	2	0
MongoDB + Nginx	1	1
Apache + Linux + MySQL + Nginx	1	0
Apache + Linux + MongoDB	1	0
Only one service	25,934	1,490
Any two services	1,619	127
Any three services	124	2
Any four services	4	0
All five services	0	0

given that web servers face direct internet exposure.

These data reveal a critical gap in practice. A typical web application stack includes Linux, a web server, and a database. Securing only one component leaves the others vulnerable to attacks. Our analysis shows that this partial hardening is unfortunately not the exception but the norm. Even among repositories implementing MySQL security hardening rules, in Ansible 94.00% fail to secure both database and OS. For web servers, the gap is even larger with over 99% of Apache and Nginx users implementing no security hardening rule.

To bridge this gap from research to practice, we developed a tool compatible with ansible-lint, the widely adopted linter for the Ansible community [23]. This tool allows developers to seamlessly integrate our semantic analysis capabilities into their existing CI/CD workflows by connecting their own LLM API keys, enabling them to scan IaC files locally to detect the categories of security issues we identified, particularly those issues that could be missed by traditional linters. We will release this tool upon publication of this paper.

V. Related Work

Static analysis for IaC security. Static analyzers and IaC linters check security using predefined rules. These rules are implemented with pattern matching, AST predicates, or policy queries. This approach is fast and deterministic. However, it misses security intent when the same hardening goal has multiple valid implementations [10], [24].

Empirical studies identify common defects and misconfigurations in IaC scripts. Rahman and colleagues conducted multiple studies on IaC security. They defined security smells for Ansible and Puppet scripts and analyzed large collections of real world scripts to identify these smells [3], [4], [25], [26]. Their work on Ansible task shows that analysis should focus on changing the effective parameter, not just the module name or string presence [27]. Sharma et al. detected configuration issues in Puppet at scale using Puppet-Lint with additional rules [5]. These studies demonstrate that rule based detection finds recurring issues. But because of the incomplete catalogs, they only capture what rules can express.

The same pattern appears beyond Ansible and Puppet. In Kubernetes, misconfigurations include issues such as exposed secrets, excessive permissions, and insecure traffic connection, among others [6]. Static checks can flag these issues, but results depend on tool coverage. In Terraform, developers struggle with alert context and actionability [28]. Syntactically correct matches can be unclear in effect when defaults and environment context matter. In Helm charts, different analyzers produce different findings because tools vary in rule scope and design [29]. Verdet et al. found that security best practice adoption in Terraform differs across cloud providers, and scanning tools support different check sets [30].

Some systems generalize smell detection across IaC languages. Saavedra et al. propose GLITCH, which translates scripts into an intermediate representation and runs detectors on the normalized form [7]. This reduces engineering effort across languages. But the approach still uses predefined smell rules. It does not verify whether a task produces an effective parameter-level security change. Opdebeeck et al. study control flow and data flow analysis in IaC smell detection [22]. They discuss the tradeoff between additional effort and practical cost. This tradeoff matters for large-scale analysis across many repositories and diverse implementation styles.

Applications of LLMs in security. Recent work applies LLMs to code generation, vulnerability detection, and automated repair. A recurring finding is that functional correctness does not guarantee security. Code can pass unit tests but contain vulnerabilities. Evaluation increasingly checks both functionality and security together [31], [32].

Code generation research shows that LLM assistants produce insecure code even when the output appears correct. Pearce et al. evaluated GitHub Copilot on tasks aligned with high risk CWE categories and found that many generated solutions contain security issues [33], while Perry et al. found that developers using AI assistants write less secure code while feeling more confident [34]. Other studies evaluated ChatGPT and similar models across programming languages and tasks and they reported similar failures [35]–[39]. During security checks and functional testing of the generated code, a large amount of vulnerable content was discovered [40], [41]. For example, code might correctly implement the requested feature but use weak cryptography or fail to validate inputs.

For vulnerability detection and repair, researchers use LLMs as code reviewers or patch generators. Results vary with prompt design and model choice. Zhou et al. surveyed LLM applications in vulnerability detection and repair and noted that performance depends heavily on prompt engineering [31]. When evaluating code, it is essential to check whether the code resolves the vulnerability, rather than simply whether the code looks similar to the reference patch [40]–[42]. Some approaches improve repair quality through model adaptation. De Fitero-Dominguez et al. fine-tuned code models on vulnerability datasets and reported better patch accuracy [43]. Charalambous et al. combined LLMs with software verification to

filter out incorrect patches [44]. These methods reduce patches that appear correct but do not fix the underlying issue. This line of work motivates LLM security evaluations that test semantic fixes rather than surface similarity [31].

VI. Conclusion

Infrastructure as Code (IaC) has become the go-to solution for managing servers in the ever-expanding modern cloud environments. Due to their inherent ability to deploy identically configured servers, they are also the appropriate medium for securing newly-provisioned infrastructure. Prior work in IaC security focused on the detection of "security smells," i.e., code patterns indicative of a vulnerability.

In this paper, we coined the term "ghost smells" (secure code patterns that should be present yet are absent) and curated a catalog of 32 popular steps for securing newly-provisioned servers and common server software. We showed that recent advances in LLMs make them an ideal mechanism for analyzing the diverse syntax of different IaC languages and developers. Moreover, we designed an analysis pipeline that allows for the efficient use of state-of-the-art LLMs to analyze IaC, while minimizing API costs. We observed that only 10.81% of Ansible repos and 4.70% of Puppet repos contain *any* server-hardening configurations, meaning that the vast majority of repos contain ghost smells. We analyzed our collected data by IaC language, by deployed software, and through a multi-service lens.

Within the repositories that contain at least one cataloged hardening rule, adoption is selective rather than comprehensive. Linux hardening dominates: in Ansible, regular system updates appear in 51.06% of these repositories, while firewall configuration appears in 33.94%; in Puppet, firewall configuration appears in 41.14% and regular updates in 11.67%. Application hardening remains uncommon even among repositories that use these services: MySQL has the highest application-service coverage, yet only 16.27% of Ansible and 7.09% of Puppet repositories that deploy MySQL contain a MySQL hardening rule. Web-server hardening is nearly absent, and multi-service hardening is limited: among MySQL users, only 6.00% of Ansible repositories and 2.14% of Puppet repositories contain both Linux and MySQL hardening rules.

All of our measurements point to limited adoption of security hardening in IaC, which translates to servers with a potentially expanded attack surface. We hope that this study can motivate the check of ghost smells in modern static analysis tools. To this end, we will be releasing an LLM-powered, IaC linter upon publication that developers and administrators can incorporate in their CI/CD workflows.

Acknowledgments

We thank the reviewers for their helpful comments. We also thank Billy Tsouvalas for his valuable feedback on this paper. This work was supported by the Office of Naval Research (ONR) under grant N00014-24-1-2193 as well as by the National Science Foundation (NSF) under grant CNS-2211575.

References

[1] "CIS Benchmarks® — cisecurity.org," https://www.cisecurity.org/cis-benchmarks, [Accessed 01-11-2025].

[2] K. Scarfone, W. Jansen, and M. Tracy, "Guide to general server security," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-123, 2008. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-123.pdf

[3] A. Rahman and L. Williams, "Source code properties of defective infrastructure as code scripts," *Information and Software Technology*, vol. 112, pp. 148–163, 2019.

[4] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 164–175.

[5] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?" in *Proceedings of the 13th international conference on mining software repositories*, 2016, pp. 189–200.

[6] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.

[7] N. Saavedra and J. F. Ferreira, "Glitch: Automated polyglot security smell detection in infrastructure as code," in *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*, 2022, pp. 1–12.

[8] "Ansible Documentation," https://docs.ansible.com/ansible/latest/index.html, [Accessed 23-10-2025].

[9] "Perforce Puppet: Infrastructure Automation Operations at Scale — puppet.com," https://www.puppet.com/, [Accessed 23-10-2025].

[10] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Information and Software Technology*, vol. 108, pp. 65–77, 2019.

[11] "OWASP Foundation," https://owasp.org/, [Accessed 01-11-2025].

[12] "ChatGPT," https://chatgpt.com/, [Accessed 19-10-2025].

[13] "Claude — claude.ai," claude.ai/, [Accessed 19-10-2025].

[14] "Google Gemini," https://gemini.google.com/, [Accessed 19-10-2025].

[15] "DeepSeek," https://chat.deepseek.com, [Accessed 19-10-2025].

[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[17] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.

[18] N. Maveli, A. Vergari, and S. B. Cohen, "What can large language models capture about code functional equivalence?" in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 6865–6903.

[19] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *International conference on data intelligence and cognitive informatics*, 2023, pp. 387–402.

[20] "New models and developer products announced at DevDay," https://openai.com/index/new-models-and-developer-products-announced-at-devday/, [Accessed 15-10-2025].

[21] "What are tokens and how to count them?" https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them, [Accessed 15-10-2025].

[22] R. Opdebeeck, A. Zerouali, and C. De Roover, "Control and data flow in security smell detection for infrastructure as code: Is it worth the effort?" in *IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 534–545.

[23] "GitHub - ansible/ansible-lint: ansible-lint checks playbooks for practices and behavior that could potentially be improved and can fix some of the most common ones for you — github.com," https://github.com/ansible/ansible-lint, [Accessed 06-11-2025].

[24] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: a survey," in *IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2022, pp. 218–225.

[25] A. Rahman and L. Williams, "Characterizing defective configuration scripts used for continuous deployment," in *IEEE 11th International conference on software testing, verification and validation (ICST)*. IEEE, 2018, pp. 34–45.

[26] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 752–764.

[27] A. Rahman, D. B. Bose, Y. Zhang, and R. Pandita, "An empirical study of task infections in ansible scripts," *Empirical Software Engineering*, vol. 29, no. 1, p. 34, 2024.

[28] H. Hu, Y. Bu, K. Wong, G. Sood, K. Smiley, and A. Rahman, "Characterizing static analysis alerts for terraform manifests: An experience report," in *IEEE Secure Development Conference*, 2023, pp. 7–13.

[29] F. Minna, F. Massacci, and K. Tuma, "Analyzing and mitigating (with llms) the security misconfigurations of helm charts from artifact hub," *Empirical Software Engineering*, vol. 30, no. 5, p. 132, 2025.

[30] A. Verdet, M. Hamdaqa, L. D. Silva, and F. Khomh, "Assessing the adoption of security policies by developers in terraform across different cloud providers," *Empir. Softw. Eng.*, vol. 30, no. 3, p. 74, 2025.

[31] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–31, 2025.

[32] S.-C. Dai, J. Xu, and G. Tao, "A comprehensive study of llm secure code generation," *arXiv preprint arXiv:2503.15554*, 2025.

[33] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," *Commun. ACM*, vol. 68, no. 2, pp. 96–105, 2025.

[34] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with ai assistants?" in *Proc. ACM SIGSAC conference on computer and communications security*, 2023, pp. 2785–2799.

[35] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by chatgpt?" in *IEEE international conference on systems, man, and cybernetics (SMC)*. IEEE, 2023, pp. 2445–2451.

[36] C. Fang, N. Miao, S. Srivastav, J. Liu, R. Zhang, R. Fang, R. Tsang, N. Nazari, H. Wang, H. Homayoun *et al.*, "Large language models for code analysis: Do {LLMs} really do their job?" in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 829–846.

[37] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, and L. C. Cordeiro, "How secure is ai-generated code: A large-scale comparison of large language models," *Empirical Software Engineering*, vol. 30, no. 2, p. 47, 2025.

[38] H. Hajipour, K. Hassler, T. Holz, L. Schönherr, and M. Fritz, "Codelmsec benchmark: Systematically evaluating and finding security vulnerabilities in black-box code language models," in *IEEE Conference on Secure and Trustworthy Machine Learning*. IEEE, 2024, pp. 684–709.

[39] A. Yildiz, S. G. Teo, Y. Lou, Y. Feng, C. Wang, and D. M. Divakaran, "Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 30 848–30 865.

[40] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana *et al.*, "Purple llama cyberseceval: A secure coding benchmark for language models," *arXiv preprint arXiv:2312.04724*, 2023.

[41] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, "Cweval: Outcome-driven evaluation on functionality and security of llm code generation," in *IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, 2025, pp. 33–40.

[42] M. L. Siddiq and J. C. Santos, "Securityeval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proc. 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022, pp. 29–33.

[43] D. de Fitero-Dominguez, E. Garcia-Lopez, A. Garcia-Cabot, and J.-J. Martinez-Herraiz, "Enhanced automated code vulnerability repair using large language models," *Engineering Applications of Artificial Intelligence*, vol. 138, p. 109291, 2024.

[44] Y. Charalambous, E. Manino, and L. C. Cordeiro, "Automated repair of ai code with large language models and formal verification," *arXiv preprint arXiv:2405.08848*, 2024.

APPENDIX A
LLM SECURITY HARDENING RULE DETECTION PROMPT

```
You are a security hardening rule detection
engine. Your single function is to parse text
and detect Linux-related, MySQL-related,
MongoDB-related, Nginx-related and Apache-
related tasks from an Ansible/Puppet script.
You cannot think, summarize, or explain. If
the input is not an Ansible/Puppet script, you
 must output NOTHING. You must ONLY output
each status line exactly as follows (each on
its own line), with "Yes" indicating presence
and "No" indicating absence:

Linux - Set up firewall: Yes/No
{All security hardening rules that will be
asked}

The following examples illustrate the specific
 actions to detect (they serve as
clarifications and are not an exhaustive list)

Action: Linux - Set up firewall
Examples:
- name: Install UFW on Debian/Ubuntu
  apt:
    name: ufw
    state: present

- name: Enable UFW firewall on Debian/Ubuntu
  ufw:
    state: enabled

- name: Install UFW on Debian/Ubuntu
  command: apt-get install -y ufw
  become: yes
{All examples that will be included}

Rules:
1. If the input is not an Ansible/Puppet
script, output NOTHING.
2. If the input is an Ansible/Puppet script,
analyze each task and detect any Linux, MySQL,
 MongoDB, Nginx and Apache tasks.
3. For each of the lines above, output "Yes"
if that file or action is detected, otherwise
"No."
4. Do NOT output any additional text,
formatting, code blocks, or explanations.

BEGIN PARSING INPUT NOW.
{Tasks that are sent to LLM}
```

APPENDIX B
TEST CASE EXAMPLES

This part provides code examples illustrating our test case types. For tasks exceeding 10 lines, we show only the critical lines that determine the test outcome.

Case 1: Single security hardening rule implementation

```
- name: Change SSH port to non-default
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?Port'
    line: 'Port 2222'
    state: present
- name: Restart SSH service
  service:
```

```
    name: sshd
    state: restarted
```

Expected: Linux - Change SSH port: Yes
Results: All models pass

Case 7: Multiple MySQL security hardening rules

```
- name: Change MySQL default port
  lineinfile:
    path: /etc/mysql/mysql.conf.d/mysqld.cnf
    line: 'port = 3307'
    insertafter: '^\[mysqld\]'
- name: Disable LOCAL INFILE
  lineinfile:
    line: 'local-infile=0'
    insertafter: '^\[mysqld\]'
- name: Remove test database
  mysql_db:
    name: test
    state: absent
- name: Remove anonymous users
  mysql_user:
    name: ''
    host_all: yes
    state: absent
```

Expected: Change default port: Yes; Disable LOCAL INFILE: Yes; Remove test databases: Yes; Remove default accounts: Yes
Results: GPT, Claude and Gemma pass, Llama partial pass

Case 2 - Default Value Trap: Sets parameter to default value

```
- name: Backup SSH config
  copy:
    src: /etc/ssh/sshd_config
    dest: /etc/ssh/sshd_config.backup
    remote_src: yes
- name: Reset SSH port to default
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^Port'
    line: 'Port 22' # Port 22 is the default
```

Expected: Linux - Change SSH port: No
Results: GPT and Claude pass, Llama and Gemma produce false positive

---

Case 4 - Irrelevant Distraction Trap: Performs unrelated operation

```
- name: Restart SSH daemon
  service:
    name: sshd
    state: restarted
    enabled: yes
```

Expected: All security hardening rules: No
Results: All models pass

Case 10 - Almost Correct Trap: Security-related keywords but ineffective configurations

```
- name: Update server tokens setting
  lineinfile:
    path: /etc/nginx/nginx.conf
    regexp: '^#\s*server_tokens'
    line: '    # server_tokens off;' # Still
    commented
- name: Configure XSS Protection (disabled)
  lineinfile:
    path: /etc/nginx/sites-available/default
    line: '    add_header X-XSS-Protection "0";' #
    Disabled
- name: Configure TLS with weak settings
  blockinfile:
    path: /etc/nginx/sites-available/default
    insertafter: 'listen 443 ssl;'
    block: |
      ssl_protocols TLSv1 TLSv1.1; # Weak protocols
      ssl_ciphers 'ALL:!aNULL:!eNULL';
```

Expected: All Nginx security hardening rules: No
Results: GPT and Claude pass, Llama and Gemma produce false positives